

**Міністерство освіти і науки України
Національний технічний університет України «КПІ» імені Ігоря Сікорського
Кафедра обчислювальної техніки ФІОТ**

**ЗВІТ
з лабораторної роботи №6
з навчальної дисципліни «Розробка мобільних застосунків»**

Виконав:

Студент 3 курсу кафедри ОТ ФІОТ,
Навчальної групи ІО-33
Шусторович Марк

Перевірив:

Професор кафедри ОТ ФІОТ
Орленко Сергій Петрович

Київ 2026

I. Мета:

змодельовати головну проблему наступного року навчання (у вигляді вибору теми дипломного проекту) шляхом самостійного формування завдання на 6 лабораторну роботу.

II. Завдання:

Написати програму під платформу Андроїд, яка буде заснована на темі, яка не розглядалась на попередніх лабораторних роботах. Самостійно сформульоване завдання - розробка ігрового застосунку у вигляді клону відомої гри Flappy Bird з нуля без використання сторонніх ігрових рушіїв. Тематика створення ігрових застосунків раніше у курсі не розглядалась.

III. Результати виконання лабораторної роботи.

Структура проєкту

Проєкт розроблено в середовищі Android Studio з використанням мови програмування Kotlin. Графічну частину гри реалізовано через компонент SurfaceView, який дозволяє виконувати рендеринг з окремого потоку у Canvas, що є оптимальним підходом для ігор з частим оновленням кадрів. Жодних сторонніх бібліотек або ігрових рушіїв не використано - все намальовано через стандартні Canvas та Paint API.

Архітектуру побудовано за класичним патерном «game loop». MainActivity.kt - точка входу, що повноекранно відображає GameView. GameView.kt - основний клас, що містить ігрову логіку, обробку дотиків та малювання. GameThread.kt реалізує окремий потік виконання game loop з цільовим FPS 60: на кожному кадрі викликаються методи update() (оновлення стану) та draw() (рендеринг). Класи Bird.kt та Pipe.kt відповідають за персонажа гравця та перешкоди - кожен інкапсулює власну логіку оновлення позиції, малювання та обробки колізій. Стан гри (меню, гра, game over) керується через прапорці у GameView, а перевірка колізій реалізована як перетин кола (птаха) з прямокутниками (трубами та землею).

Результат тестування

Після запуску застосунок відкривається у повноекранному режимі та одразу показує стартовий екран з назвою гри, надписом «Тапніть, щоб почати» та поточним рекордом. При першому дотику до екрана гра починається - птах робить стрибок вгору та починає падати під дією гравітації, а на екрані з'являються труби з проміжками випадкової висоти.

Перевірено основну ігрову механіку: кожен дотик до екрану викликає стрибок птаха, інакше він падає під дією гравітації з обмеженням максимальної швидкості. Труби рухаються справа наліво зі сталою швидкістю та з'являються з рівномірним інтервалом. При успішному проходженні труби рахунок збільшується на 1 та відображається великим шрифтом у верхній частині екрану. У разі зіткнення птаха з трубою або з землею гра переходить у стан Game Over, де відображається фінальний рахунок та рекорд, після чого

тап перезапускає гру.

Перевірено коректність обробки життєвого циклу Activity: при згортанні застосунку (onPause) game loop призупиняється, при поверненні (onResume) - відновлюється без втрати поточного стану. Рекорд зберігається протягом сесії гри. Швидкість анімації стабільна, кадрова частота тримається близько 60 FPS навіть при наявності декількох труб на екрані одночасно. Гра коректно працює на пристроях з різною роздільною здатністю, оскільки всі координати обчислюються відносно розмірів екрану.

Скріншоти роботи застосунку



Рис. 1. Стартовий екран гри

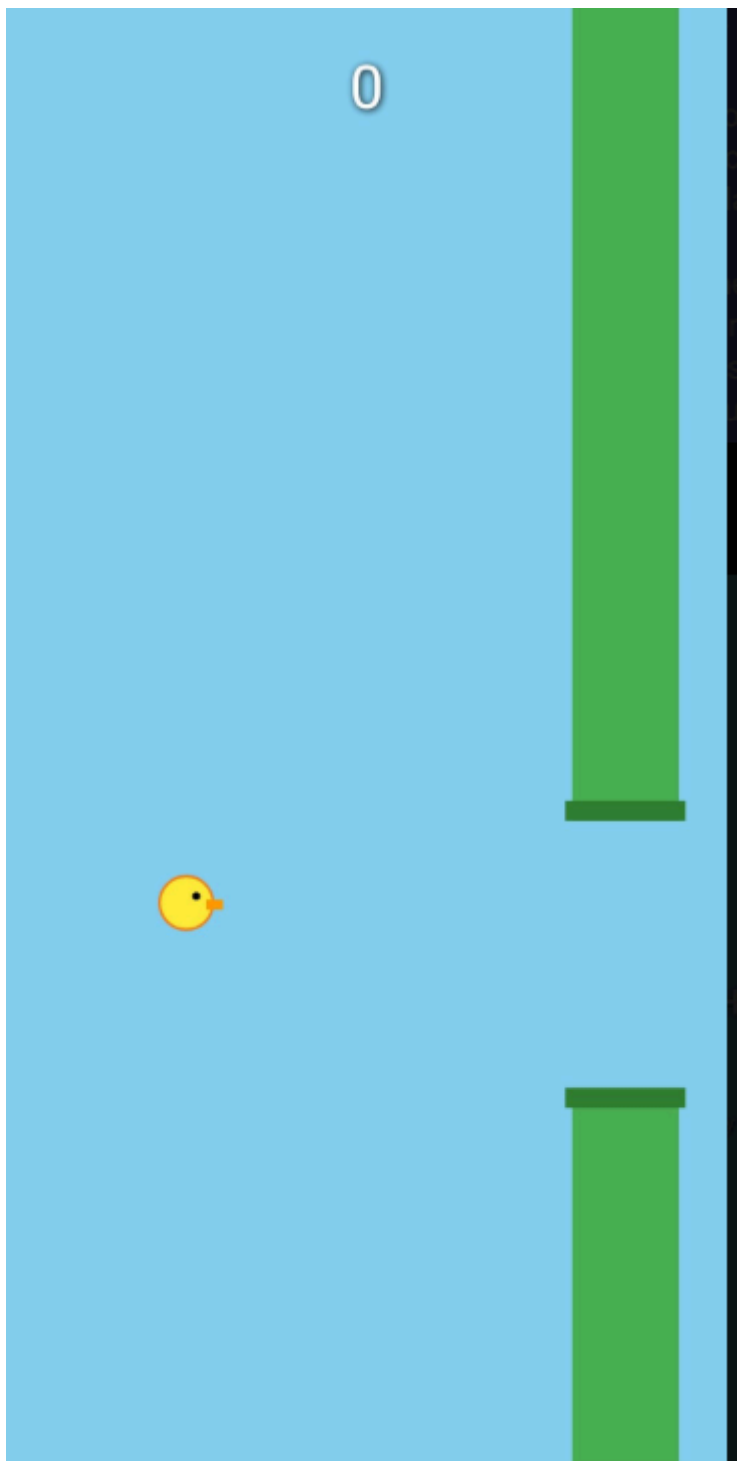


Рис. 2. Ігровий процес з птахом та трубами

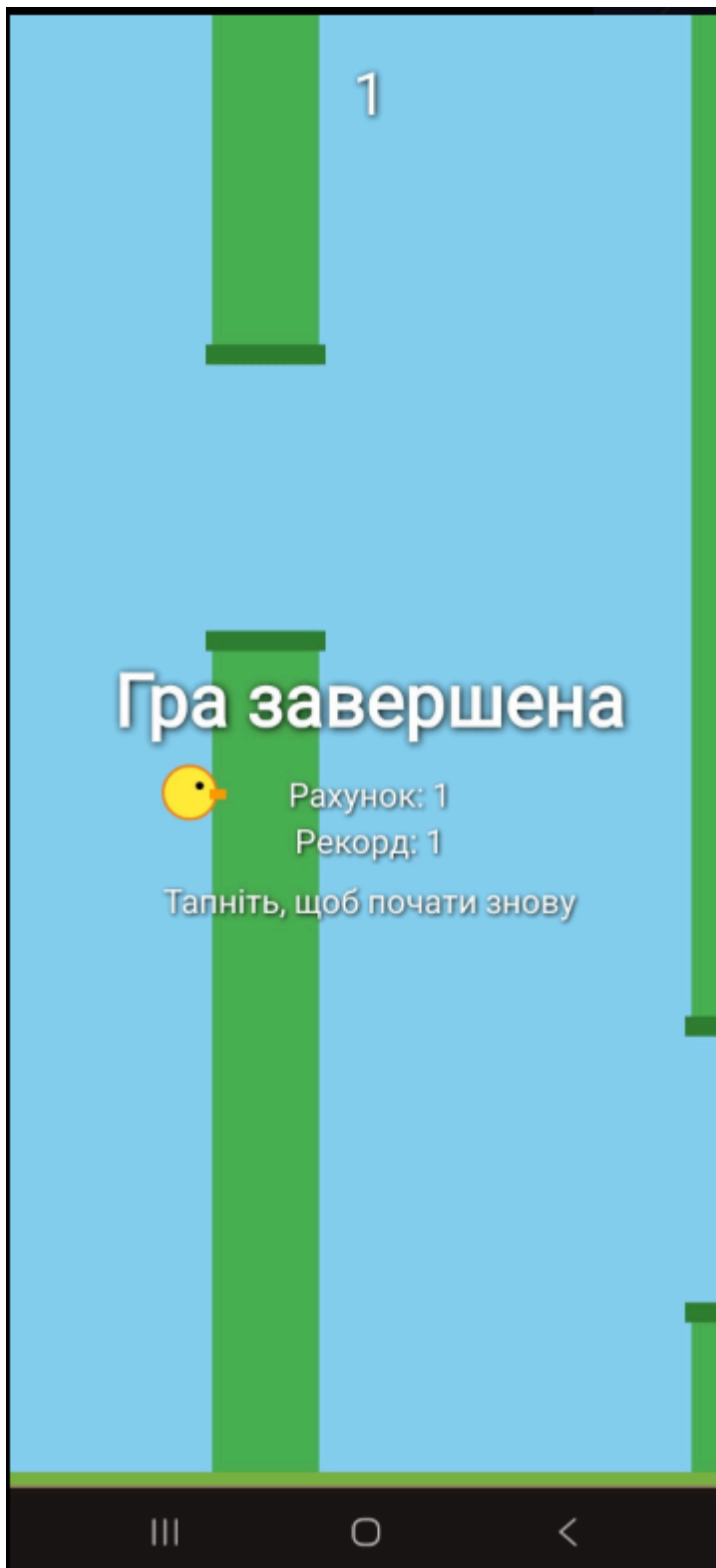


Рис. 3. Екран Game Over з результатом гри

IV. Висновки.

В результаті виконання лабораторної роботи було розроблено повноцінний ігровий застосунок під платформу Android - клон гри Flappy Bird, реалізований з нуля без використання сторонніх ігрових рушіїв. Засвоєно роботу з SurfaceView для високопродуктивної 2D-графіки, реалізацію game loop в окремому потоці виконання, побудову фізики гри (гравітація, імпульси) та системи обробки колізій. Також опрацьовано

керування станами гри (меню, активна гра, кінець гри) та коректну обробку життєвого циклу Activity у багатопоточному середовищі. Усі вимоги до самостійно сформульованого завдання виконано в повному обсязі.